

Algorithmes et Pensée Computationnelle

Algorithmes spatiaux

Le but de cette séance est de comprendre les caractéristiques des données spatiales et les structures de données couramment utilisées pour les manipuler. Lors de cette séance, nous implémenterons des algorithmes de manipulation de structures de données spatiales présentés en cours. Au terme de la séance, l'étudiant sera en mesure d'utiliser plusieurs algorithmes spatiaux pour résoudre des problèmes de base de façon efficace.

1 Nearest-Neighbor

Dans cette section, nous allons implémenter une recherche du plus proche voisin. Le but de cette méthode est de trouver le voisin le plus proche du point de départ en tenant compte de ce point de "départ" et un ensemble de points. Nous allons implémenter cet algorithme et ensuite l'étendre à un algorithme des "k plus proches voisins" (recherche des k voisins les plus proches plutôt que du seul voisin le plus proche). Nous vous recommandons de traiter les questions dans l'ordre.

Question 1: (🕒 5 minutes) Python — La fonction de distance

Pour implémenter notre recherche, nous avons besoin d'écrire une fonction permettant de calculer la distance entre 2 points. Ecrivez une fonction qui permet de calculer la distance entre 2 points.

💡 Conseil

Soit 2 points en 2 dimensions (x_1, y_1) et (x_2, y_2) , la distance euclidienne entre ces 2 points est donnée par : $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. En Python, la fonction permettant de faire une racine carrée est `sqrt` de la librairie `math`. Pour mettre au carré, on utilise `nombre**2`.

>_ Solution

```
1 import math #permet d'importer la librairie nécessaire au calcul de la
  racine carrée
2
3 def calculate_distance(point1, point2):
4     #Cette fonction retourne la distance euclidienne entre 2 points
5     return math.sqrt((point2[0]-point1[0])**2 + (point2[1]-point1[1])**2)
```

Note : Vous pouvez utiliser `**0.5` au lieu de la fonction `math.sqrt()`.

Question 2: (🕒 10 minutes) Python — Nearest-neighbor search

Implémentez la recherche du voisin le plus proche. Cette dernière fonctionne de la façon suivante :

1. Créez une fonction qui prend comme arguments une liste de points et un point de départ.
2. Gérez le cas où la liste de points est vide.
3. Traversez chaque point.
4. Pour chaque point, calculez la distance entre ce point et le point de départ.
5. Retournez un tuple contenant les coordonnées du point le plus proche sous forme d'un tuple et la distance — par exemple : $((x, y), dist)$.

💡 Conseil

Utilisez la fonction de distance de la Question 1 et parcourez les points à l'aide d'une boucle `for`. Si votre input est `[(2, 3), (5, 6), (1, 4), (2, 4), (3, 5)]` et que le point de départ est `(4, 4)`, alors l'output devra être `((3, 5), 1.414)`. Ici, `(3, 5)` sont les coordonnées du point le plus proche et `1.414` est la distance euclidienne entre notre point de départ et le point le plus proche.

Note : Pour que votre programme fonctionne, écrivez votre algorithme du plus proche voisin dans le même programme que celui de la Question 1.

>_ Solution

```
1  import math #permet d'importer la librairie nécessaire au calcul de la
    racine carrée
2
3  # Question 1
4  def calculate_distance(point1,point2):
5      #Cette fonction retourne la distance euclidienne entre 2 points
6      return math.sqrt((point2[0]-point1[0])**2+(point2[1]-point1[1])**2)
7
8  # Question 2
9  def nearest_neighbor(start, point_set): # start correspond au point
    de départ, point_set correspond à l'ensemble des points
10     if len(point_set) == 0: # le cas où la liste de points est vide
11         return None, None
12
13     min_distance = calculate_distance(start, point_set[0]) # on
    initialise la distance minimale avec la valeur de la distance
    entre start et point_set[0]
14     nearest_nei = point_set[0] # on initialise le point le plus
    proche avec la valeur du premier point
15
16     for i in range(len(point_set)): # on parcourt tous les points de
    l'ensemble
17         distance = calculate_distance(start, point_set[i])
18         if distance < min_distance:
19             min_distance = distance
20             nearest_nei = point_set[i]
21
22         # Cette partie du code détermine si le point actuellement
    considéré, est plus proche du point de départ que les points
23         # parcourus jusqu'ici. Si c'est le cas, on redéfinit la
    distance minimale et on "enregistre" les coordonnées du point
24
25     return nearest_nei, min_distance # ou (nearest_nei, min_distance)
26
27
28 if __name__ == '__main__':
29     a = [(2, 3), (5, 6), (1, 4), (2, 4), (3, 5)] # Liste de points
30     b = (4, 4) # Point de départ
31
32     point, distance = nearest_neighbor(b, a)
33     print(f'{point}, {distance}')
34     # Devrait retourner (3, 5), 1.4142135623730951
```

Question 3: (🕒 15 minutes) K-nearest-neighbor search

Améliorez l'algorithme du voisin le plus proche effectué plus haut afin qu'il puisse retourner des *K*-plus

proches voisins.

💡 Conseil

Appliquez l'algorithme du plus proche voisin K -fois. À la fin de chaque itération, retirez le voisin le plus proche de l'ensemble des points sur lequel l'algorithme s'applique. De cette façon, vous trouverez le second voisin le plus proche, le troisième, etc...

Votre fonction devra retourner une liste sous la forme : $[(point1, distance1), (point2, distance2), \dots]$. En considérant un input $[(2, 3), (5, 6), (1, 4), (2, 4), (3, 5)]$, un point de départ $(4, 4)$ et un nombre de voisins $K = 2$, l'output de votre algorithme devra être : $[((3, 5), 1.4142135623730951), ((2, 4), 2.0)]$. Attention au type des variables que vous manipulez. Pour rappel, les tuples sont immuables en Python. Pensez à créer de nouveaux tuples contenant les coordonnées des points et leurs distances.

Note : Pour que votre programme fonctionne, écrivez votre algorithme dans le même programme que celui de la Question 1 et la Question 2.

>_ Solution

```
26 # Question 3
27 def K_nearest_neighbor(start, point_set, K):
28     k_nearest_nei = []
29
30     for j in range(min(K, len(point_set))): # A chaque itération, on
        applique l'algorithme du nearest neighbor mais sur un ensemble de
        points réduit
31         point, distance = nearest_neighbor(start, point_set)
32         k_nearest_nei.append((point, distance))
33         if point is not None: # si K est plus grand que le nombre de
            points dans point_set, on évite une erreur en vérifiant que point
            n'est pas None
34         point_set.remove(point) # On retire de l'ensemble de
            points le voisin le plus proche, de cette manière, à chaque
            itération,
            # le voisin le plus proche sera de plus en plus éloigné.
35
36
37     return k_nearest_nei
38
39 if __name__ == '__main__':
40     a = [(2, 3), (5, 6), (1, 4), (2, 4), (3, 5)] # Liste de points
41     b = (4, 4) # Point de départ
42
43     k_nearest_neighbors = K_nearest_neighbor(b, a, 2)
44     print(k_nearest_neighbors)
45     # Devrait afficher [((3, 5), 1.4142135623730951), ((2, 4), 2.0)]
46     for point, distance in k_nearest_neighbors:
47         print(f'{point}, {distance}')
48     # Devrait afficher (3, 5), 1.4142135623730951 et (2, 4), 2.0
```

2 K-dimensional tree

Question 4: (🕒 5 minutes) Lecture du code — Construction d'un KD-Tree

Algorithm 1 BuildKDTree(points, depth, k)

```
1: if points =  $\emptyset$  then
2:   return NIL
3: end if
4: axis  $\leftarrow$  depth %  $k$                                 ▷ Select axis based on current depth
5: points  $\leftarrow$  SORT(points, key = axis)                 ▷ Sort by current axis
6: median_idx  $\leftarrow$  INTEGER(length(points)/2)           ▷ Index of median list item
7: median  $\leftarrow$  KDNode(points[median_idx])
8: left  $\leftarrow$  points[1 ... median_idx - 1]
9: right  $\leftarrow$  points[median_idx + 1 ... length(points)]
10: median.left  $\leftarrow$  BuildKDTree(left, depth + 1,  $k$ )
11: median.right  $\leftarrow$  BuildKDTree(right, depth + 1,  $k$ )
12: return median
```

💡 Conseil

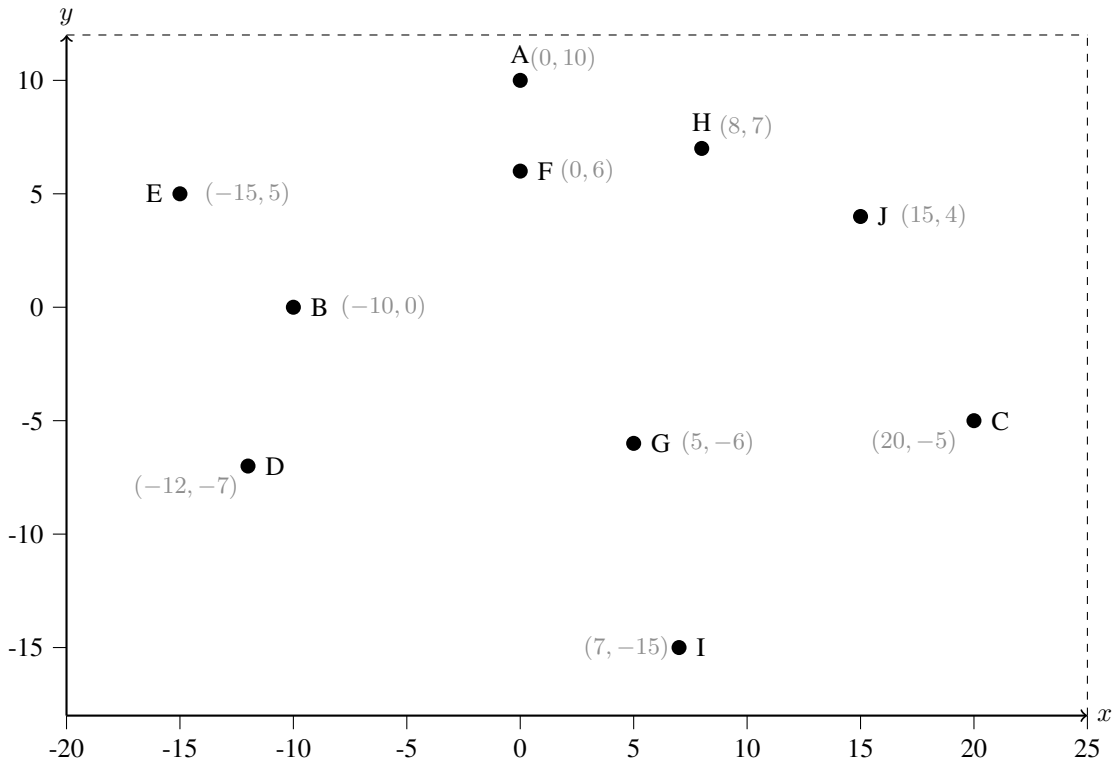
Cet algorithme construit récursivement un arbre KD (KD-Tree) à partir d'une liste de points de dimension k . À chaque appel, il sélectionne d'abord l'axe de séparation en fonction de la profondeur courante (modulo le nombre de dimensions), puis trie les points selon cet axe et choisit le point médian comme racine du nœud courant afin de garantir un arbre équilibré. Les points situés avant ce médian constituent le sous-ensemble gauche et ceux après le médian le sous-ensemble droit. L'algorithme est appelé récursivement ensuite sur chacun de ces deux sous-ensembles pour construire respectivement les sous-arbres gauche et droit. Le processus se poursuit jusqu'à ce qu'il n'y ait plus de points à insérer.

>_ Solution

Voir la démonstration des TP pour la visualisation de la construction d'un arbre KD.

Question 5: (🕒 10 minutes) KD-Tree, un échauffement : Papier

Vous trouverez ci-dessous une liste de points numérotés de 1 à 10. Placez-les dans un KD-Tree et dessinez la séparation de l'espace qui en résulte.

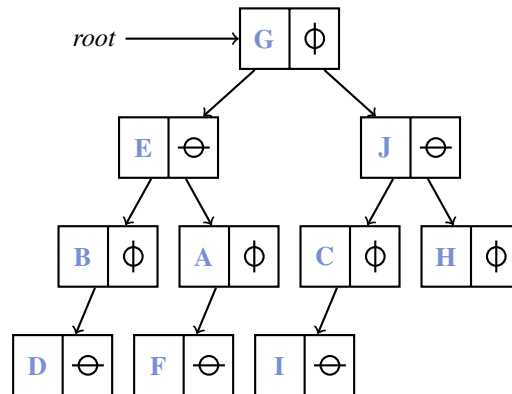


Conseil

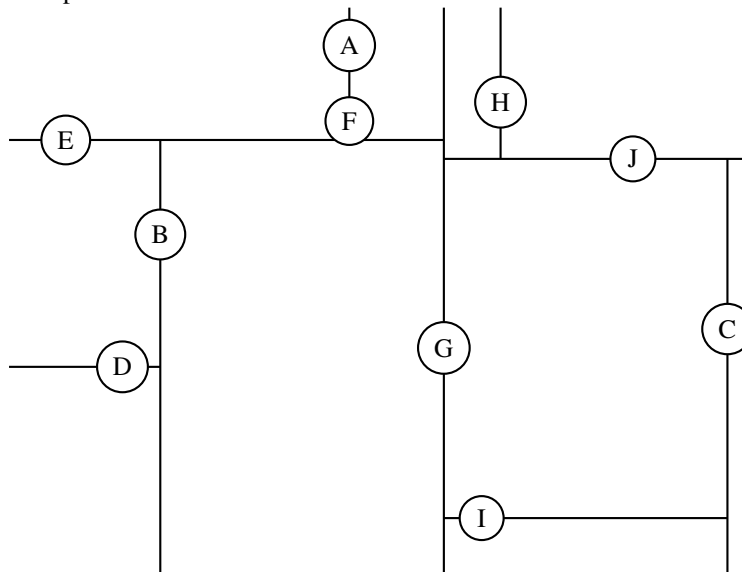
La première division se fait de façon verticale. Veillez à bien insérer le point milieu (median) dans chaque étape. Les nœuds se situant au même niveau devraient diviser l'espace selon le même axe.

>_ Solution

Voici le KD-Tree :



La division de l'espace suivant le KD-Tree :



Question 6: (🕒 15 minutes) KD-Tree : Python

L'objectif de cet exercice est d'écrire une fonction permettant d'ajouter un nœud à un KD-Tree. Les nœuds sont sous la forme $[(x, y), \text{enfant à gauche}, \text{enfant à droite}]$, x et y étant les coordonnées du nœud considéré. Chaque enfant peut être soit un nœud ou une feuille. Complétez le code contenu dans le fichier `Question6.py`.

Voici le pseudo-code permettant d'ajouter un nœud à un KD-Tree :

Algorithm 2 AddKDTree(node, point, k , cutaxis)

```
1: if node = NIL then
2:   new_node  $\leftarrow$  KDNode(point)
3:   return new_node
4: end if
5: if point[cutaxis]  $\leq$  node.point[cutaxis] then           ▷ Insert into the left (lesser) subtree
6:   new_cutaxis  $\leftarrow$  (cutaxis + 1) (mod  $k$ )
7:   node.left  $\leftarrow$  AddKDTree(node.left, point,  $k$ , new_cutaxis)
8: else
9:   new_cutaxis  $\leftarrow$  (cutaxis + 1) (mod  $k$ )
10:  node.right  $\leftarrow$  AddKDTree(node.right, point,  $k$ , new_cutaxis)
11: end if
12: return node
```

⚠ Attention

Afin de simplifier la question, vous ne devez pas créer une classe KDNode pour représenter les nœuds. Chaque nœud est représenté comme une liste. Donc, quand vous transformez le pseudocode, node.point sera node[0], node.left sera node[1], et node.right sera node[2].

💡 Conseil

Si votre réponse est correcte, votre programme devrait afficher : [(0, 10), [(-10, 0), None, None], None]. Pour rappel, en Python, NIL correspond à None. Dans le pseudocode, KDNode permet de créer un nouveau nœud. Vous pouvez vous inspirer de la structure de main définie dans le fichier Question6.py.

>_ Solution

```
1 # Question 6
2
3 def add_node(node, point, k, cutaxis=0):
4     # Si le noeud n'existe pas, nous sommes donc dans une feuille, et
5     # il faut créer le noeud
6     if node is None:
7         node = [point, None, None]
8         return node
9
10    # (cutaxis + 1) % k ensures correct cycle 0 -> 1 -> 0 -> 1...
11    next_axis = (cutaxis + 1) % k
12
13    if point[cutaxis] <= node[0][cutaxis]:
14        node[1] = add_node(node[1], point, k, next_axis)
15    else:
16        node[2] = add_node(node[2], point, k, next_axis)
17
18    return node
```

>_ Solution

```
19 if __name__ == "__main__":
20     # Nous définissons ici juste la racine de l'arbre
21     root = [(0,10), None, None]
22     k = 2 # Nous travaillons en 2 dimensions
23
24     # Point que nous voulons ajouter dans l'arbre
25     point = (-10,0)
26
27     add_node(root, point, k)
28     print(root)
```

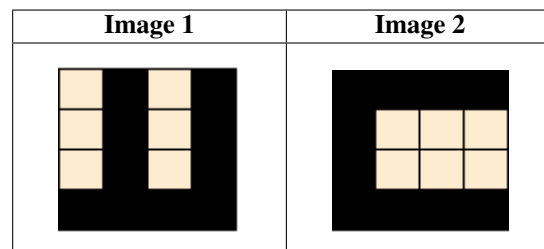
>_ Solution

Si la coordonnée du point à ajouter est inférieure à celle du nœud selon l'axe de découpe en considération, alors le point doit se trouver dans le sous-arbre de gauche. Par convention, le nœud de gauche correspond dans la liste [(x,y), nœud de gauche, nœud de droite] à l'indice 1, par conséquent, on appelle la fonction de façon récursive pour ajouter le point, mais cette fois-ci en partant d'un cran plus bas dans l'arbre. Cela se répète jusqu'à ce qu'on ait atteint les feuilles et qu'un nouveau nœud doive être créé.

3 Quad-Tree

Question 7: (🕒 10 minutes) **Une mise en train : Papier**

Encodez les images ci-dessous dans un Quad-Tree.



💡 Conseil

Pour réussir cet exercice vous devez diviser chaque nœud en 4 sous-espaces (le plus petit sous-espace étant un carré) de taille égale, et ce autant de fois que nécessaire. La branche la plus à gauche (en haut) correspond au quadrant NW puis en allant de gauche à droite et puis en bas (comme dans le sens de lecture) : NE, SW, SE.

Indice : Votre arbre devrait avoir une profondeur de 2 et disposer de 16 feuilles.

>_ Solution

Image 1 :

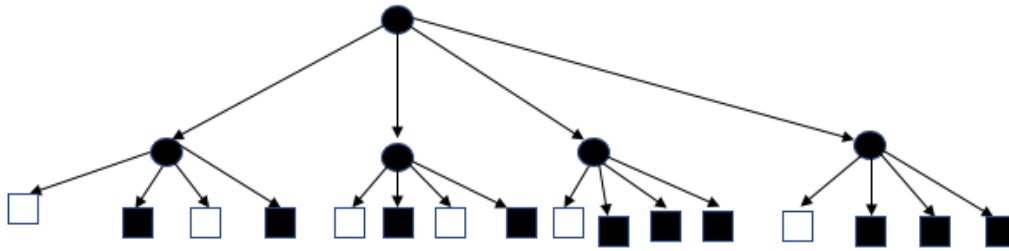
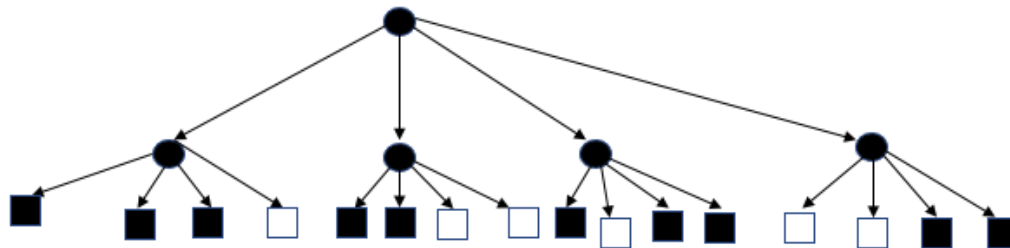
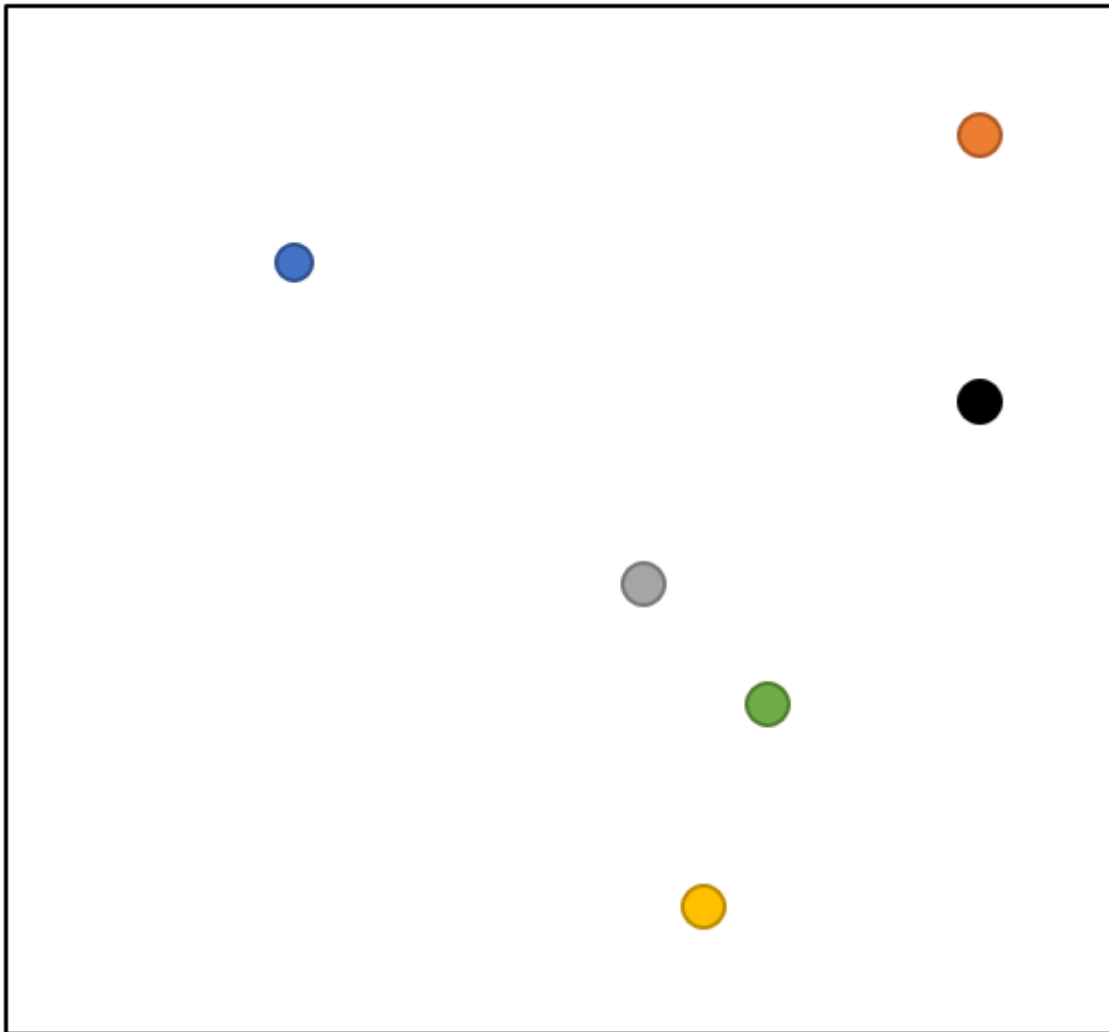


Image 2 :



Question 8: (🕒 10 minutes) Une mission capitale : Papier Optionnel

Récemment embauché par la CIA, vous êtes à la recherche d'un individu se cachant dans une des villes suivantes : Bleu, Orange, Noir, Gris, Vert et Jaune. Votre mission, si vous l'acceptez, est de créer un Quad-Tree qui vous permettra de géolocaliser le criminel de façon efficace. Vous trouverez ci-dessous une carte de villes. Créez le Quad-Tree et rétablissez la justice.



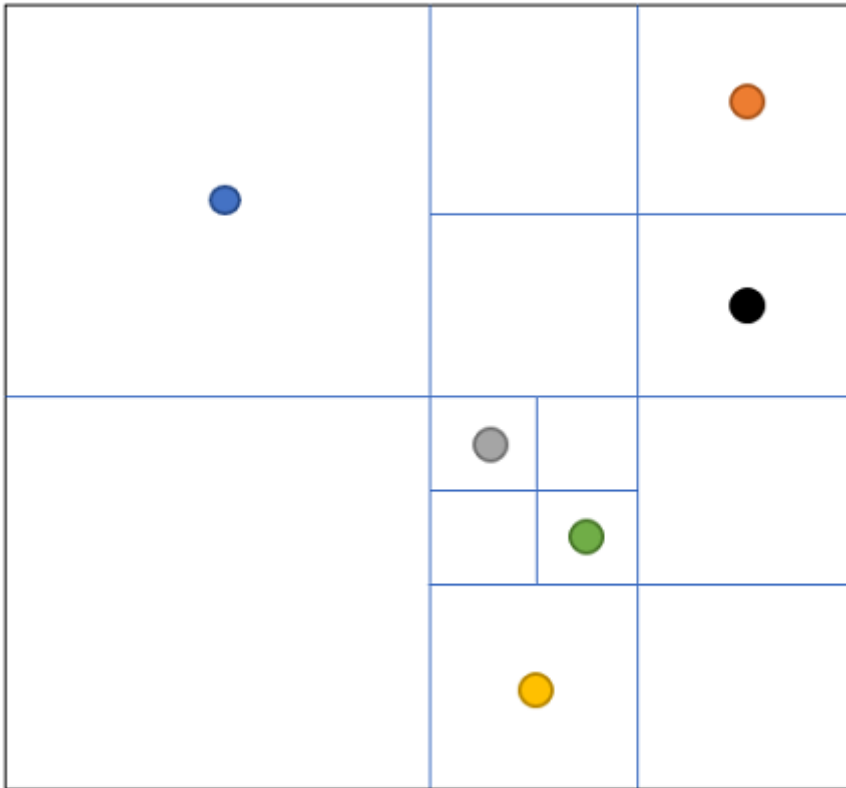
 Conseil

Commencez par diviser la carte de la ville de façon adéquate puis construisez le graphe.

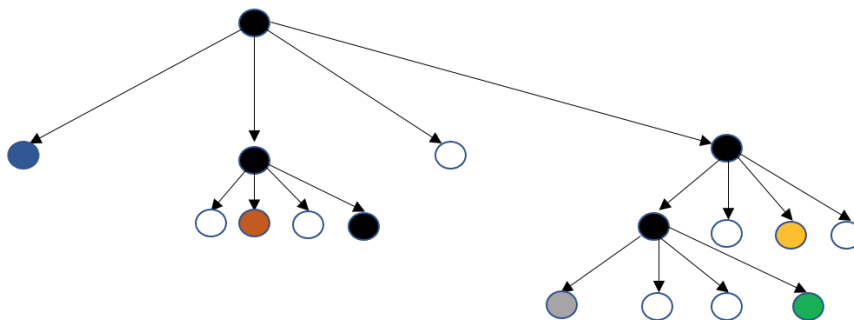
Remarque : Les différentes branches de l'arbre n'auront pas toutes la même profondeur.

>_ Solution

Voici la division de la carte qui permet de construire le Quad-Tree :



Le Quad-Tree qui en résulte :



Note : Un rond blanc correspond à un quadrant vide.